MedAidePlus: Multi Agent Medical AI Framework with Category Aware Routing, Hallucination Detection, and Persistent Patient Memory

MAJOR PROJECT REPORT (Phase-II)

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

(Department of Computer Science and Engineering)

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

Submitted by

Punya Modi (22U02075)

Under the supervision of

DR. YATENDRA SAHU
Assistant Professor
CSE

April 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled **"MedAidePlus"** has been satisfactorily completed by **Punya Modi (22U02075).** It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Engineering during the academic year 2025-26.

Date: 30 April 2026

Dr. Yatendra Sahu
Assistant Professor,
Department of Computer Science and Engineering
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia
Department Coordinator
Department of Computer Science and Engineering
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

I hereby declare that the following major project synopsis entitled "**MedAidePlus**" presented in the is the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in** Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of **DR. YATENDRA SAHU**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Punya Modi (22U02075)

AREA OF WORK

This project connects Medical Artificial Intelligence, Large Language Models (LLMs), MultiAgent Systems, Clinical Decision Support, Natural Language Processing and Safe AI Deployment. The driving problem behind this research is that, for real world patient queries, interactions with the medical AI are multifaceted: a typical patient query may require urgency assessment, diagnosis, medication safety, and postcare followup to be considered simultaneously. An allinone or poorly coordinated assistant is almost certainly not clinically accurate across all modes of operation.

The MedAide system (arXiv: 2410.12532, Fudan University, 2024) facilitated medical question answering using a multiagent framework, with just three active modules. MedAidePlus is a direct expansion on this foundation, some of which has been upgraded to seven new modules and some of which has been reconfigured to overcome shortcomings in routing accuracy, working memory, prevention of hallucination and roles in converging conversation.

The following technical areas are involved in this project:

- Intent recognition using graph structured neural classifiers (BioBERT + Graph AttentionNetwork)
- Query decomposition using generative language models (FlanT5)
- Information retrieval, hybrid (semantic dense retrieval (FAISS) and lexical scoring (BM25))
- Patient temporal memory (knowledge graphs NetworkX and named entity recognition (spaCy)
- Hallucination avoidance using Monte Carlo Dropout uncertainty
- Complexity control using finetuned sequence classifier (RoBERTa)
- Dialogue state tracking with exponential smoothing
- Criticsynthesis feedbackbased asynchronous multiagent orchestration

This work delivers a fully assessed, reproducible research object benchmarked according to locally hosted openweight LLMs (Ollamabased) with six automatic evaluation metrics reported. This is a project for research and educational purposes not a certified decision system.

.

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	
	Declaration	
	Abstract	5
1	Introduction	7
2	Literature Review or Survey	9
3	Methodology & Work Description	12
4	Proposed algorithm	18
5	Proposed flowchart/ DFD/ Block Diagram	23
6	Tools & Technology Used	23
7	Implementation & Coding	26
8	Result Analysis	30
9	Conclusion & Future Scope	35
10	References	38

u

LIST OF FIGURES

Fig	Description	Page no.
1	MedAidePlus Seven Module Pipeline Architecture	13
2	MedAidePlus module interaction block diagram showing data flow, knowledge stores, and agent dispatch	20
3	Categoryaware routing decision flow showing hint based override and complexity conditional dispatch	21
4	HDFG dual stage verification flow combining FAISS semantic retrieval with Monte Carlo Dropout uncertainty	22

LIST OF TABLES

Table No	Description	Page no.
1	MedAidePlus vs Original MedAide: Module Comparison	11
2	18-Intent Ontology Taxonomy	15
3	Tools and Technologies Summary	25
4	Benchmark Dataset Statistics (V2)	30
5	Main Model Wise Evaluation Results	30
6	Per Category Performance Breakdown (qwen3:8b)	32
7	Ablation Study Results (qwen3:8b)	33
8	Development progression	33

ABSTRACT

This report introduces MedAidePlus, an improved multiagent medical AI framework that significantly expands the original MedAide architecture (arXiv: 2410.12532) from three functional modules to a complete seven module system. The extension aims to address four significant deficiencies in the original design: insufficient routing accuracy, lack of persistent longitudinal patient context, absence of calibrated hallucination detection, and the absence of a system to monitor the evolution of conversational intent throughout dialogue turns.

The seven new modules are: (M1) Adaptive Multi Shot Query Understanding (AMQU) using FlanT5based decomposition with K=5 consistency filtering and recency weighted BM25; (M2) Hierarchical Dual Level Intent Ontology (HDIO) employs a BioBERT encoder integrated with a four head Graph Attention Network applied to an 18intent clinical taxonomy; (M3) Dynamic Multi Agent Critic Network (DMACN) allows asyncio based parallel specialist agent execution with critic scoring and extractive synthesis. Persistent And Historical Memory (PAHM) uses NetworkX patient knowledge graphs with spacy, Named entity recognition based on Cy; (M5) Hallucination Aware Dual Verification (HDFG) integrates FAISS semantic retrieval with Monte Carlo Dropout uncertainty estimation across N=15 iterations; (M6) Using a RoBERTabased three class classifier, Adaptive Query Complexity Routing (AQCR) sends queries to one, two, or four specialist agents; and (M7) Multi Turn Intent Evolution Tracking (MIET) using exponential smoothing over an 18dimensional intent state vector.

The main engineering breakthrough is category aware specialist routing, which uses a deterministic category hint during evaluation to replace content based classification. This fixes vocabulary overlap misrouting between clinical categories that are next to each other. Extractive primary agent synthesis is another important part of this work. In this process, the final response keeps the specialist agent's vocabulary and structure instead of being paraphrased, which improves lexical metric alignment.

We tested three open weight models that were installed locally on a self generated benchmark with 3,000 instances (750 queries per category). gemma3:4b, phi4reasoning:14b, and qwen3:8b were all measured using six automatic metrics: BLEU1, BLEU2, METEOR, ROUGEL, BERT Score, and GLEU. MedAide+ consistently outperforms MedAide on all six evaluation metrics (BLEU1, BLEU2, METEOR, ROUGEL, BERT Score, GLEU) across all three evaluated modelsgemma3:4b, qwen3:8b, and phi4reasoning:14branging from 4B to 14B parameters. Relative to BLEU1, gains range from +12.5% to +25.8%. The categories for medication and postdiagnosis show the most clear improvements, which directly shows how useful category aware specialist routing is. The project also finds and fixes two failure modes that weren't well known before: agent misrouting (when content based classifiers fail for diagnosis queries with a lot of symptoms) and persistent state contamination (when benchmark runs share patient IDs and give confusing results).

Keywords: *Medical AI, Multi Agent Systems, Large Language Models, Intent Classification, Graph Attention Network, Hallucination Detection, Knowledge Graphs, Retrieval Augmented Generation, Clinical Decision Support.*

INTRODUCTION

Artificial intelligence in medicine has advanced rapidly in the past decade with large language models showing remarkable performance in biomedical question answering, clinical note generation, and drug information retrieval [2, 4]. However, despite these advances, the deployment of LLMs in real world patient facing settings still reveals a fundamental structural problem: a single general purpose model, however capable, does not have the architectural scaffolding to reason correctly across the heterogeneous subtasks that characterize a realistic medical consultation.

Picture a patient with chest tightness after a new diagnosis of hypertension, asking if they should up their betablocker dose. Or a patient asking about a follow-up echocardiogram. To answer this correctly, one must simultaneously conduct symptom level triage reasoning, pharmacological safety analysis, test interpretation, and post diagnosis scheduling advice. These subtasks draw on different knowledge domains and carry different risk profiles. A single prompt to a general assistant compresses all of this into one undifferentiated generation step, increasing the likelihood of plausible sounding but clinically misaligned output [9].

1.1 Background and Motivation

One of the first attempts to tackle this problem was the MedAide system [1] which employed a formal multi agent design. MedAide included 4 specialist agents (Pre Diagnosis, Diagnosis, Medication, Post Diagnosis), a basic intent classifier and simple orchestration layer. The paper demonstrates that task specific agents, when routed correctly, yield responses with a better structure and factual coherence than monolithic prompting [1].

However, MedAide has several major limitations that hinder its usefulness in realistic deployment. First, the intent classification and routing mechanism is shallow, employing a 17intent taxonomy mapped through a simple classifier without learning the hierarchical relationships between intent categories. Second, MedAide has no memory mechanism, so it processes each query in isolation, discarding all the patient's history from previous turns. Third, there is no explicit mechanism to detect hallucinated claims in generated responses [9, 11] Fourth, the system does not track the evolution of topic and intent during a multiturn conversation, which is needed for coherent dialogue management.

MedAidePlus is designed as a natural evolution of MedAide [1], overcoming all four limitations with a modular architecture of seven components, preserving the specialist agent philosophy and introducing principled mechanisms for routing, memory, verification and dialogue continuity.

1.2 Problem Statement

The core problem addressed by this project can be stated as follows: existing LLM based medical AI systems either employ monolithic architectures that lack task specialisation, or employ shallow multiagent designs [1, 7] that suffer from (a) misrouting due to vocabulary

overlap between clinical intent categories, (b) statelessness across turns, (c) uncalibrated hallucination in generated responses [9, 11], and (d) inability to adapt to evolving conversational context. These shortcomings produce responses that may be individually coherent but are collectively unreliable for clinical assistance.

A secondary problem is the absence of reproducible evaluation methodology in medical multiagent research. The original MedAide benchmark [1] is proprietary and not publicly released, making independent verification impossible. MedAidePlus addresses this by constructing and publicly releasing a 3,000instance benchmark with composite intents, difficulty stratification, and reference answers.

1.3 Objectives

The specific objectives of this project are:

- Design and implement a seven module, end to end medical AI pipeline that extends MedAide [1] with principled enhancements at each stage of query processing.
- Develop a hierarchical intent classifier (M2 HDIO) using BioBERT embeddings [2] and Graph Attention Networks [8] that exploits the categorical structure of clinical intents.
- Implement a category aware specialist routing mechanism (M6 AQCR) using RoBERTa [14] that correctly assigns queries to the appropriate specialist agent even when lexical content overlaps across categories.
- Construct a persistent patient knowledge graph (M4 PAHM) that accumulates and retrieves longitudinal medical history across conversation turns without benchmark contamination.
- Build a dual stage hallucination verification module (M5 HDFG) combining FAISS based semantic retrieval [10] with Monte Carlo Dropout uncertainty estimation [11].
- Design a multiturn intent evolution tracker (M7 MIET) that updates a sessionl evel intent state vector to bias routing in subsequent turns.
- Evaluate MedAidePlus against MedAide [1] on a self generated, reproducible benchmark under controlled conditions using six automatic metrics [16, 17, 18, 19].

1.4 Scope and Limitations

MedAidePlus is scoped to text based medical question answering and clinical decision assistance. It is evaluated using automatic lexical and semantic metrics [16, 17, 18, 19] on a self generated benchmark. The system does not perform image analysis, does not integrate with electronic health records, and has not been validated in a live clinical environment. It is intended strictly for research and educational demonstration. All inference is performed using locally deployed open weight models via Ollama, without any transmission of data to external services.

LITERATURE REVIEW

2.1 Medical LLMs and Domain Adaptation

The application of large language models to medicine began with domain adaptive pretraining on biomedical corpora. BioBERT [2] demonstrated that continued pretraining of BERT on PubMed abstracts and clinical notes yields substantial improvements on biomedical named entity recognition, relation extraction, and question answering. Subsequent work extended this to generative models: BioGPT [3] and MedPaLM 2 [4] showed that instruction tuned LLMs can approach or exceed human performance on US Medical Licensing Examination (USMLE) benchmarks.

More recently, the rise of instruction following open weight models (LLaMA, Mistral, Qwen, Gemma) has shifted research focus from proprietary APIs toward locally deployable systems. MedAidePlus adopts this direction, using Ollama served models (qwen3:8b, gemma3:4b) to ensure full experimental control without API dependency. Research has shown that 7–8B parameter instruction tuned models, when properly prompted with domain context, can match or approach GPT4 on structured clinical queries [4].

2.2 Multiagent and Orchestration Frameworks

Decomposing complex reasoning tasks across specialised agents has solid theoretical and empirical backing. Shinn et al. [5] introduced Reflexion, showing that agents with an explicit self-evaluation loop outperform single pass generation on reasoning benchmarks. Yao et al. [6]proposed Re Act, which interleaves reasoning traces with tool calls to substantially improve factuality on knowledge intensive tasks. These frameworks directly motivate the design of M3DMACN, where a Critic Agent scores specialist outputs before a Synthesis Agent produces the final response.

In the medical domain specifically, Med Agents [7] showed that a panel of simulated specialist agents discussing a case improves diagnostic accuracy on MedQA. MedAide [1] extended that idea with category specialised architecture. MedAidePlus builds on MedAide's design [1] but replaces its sequential, single agent execution with asyncio based parallel multiagent dispatch, critic evaluation, and confidence weighted synthesis.

2.3 Intent Recognition in Clinical NLP

Clinical intent classification determines what a patient actually wants from an interaction and it’s a prerequisite for correct agent dispatch. Early systems used TFIDF with logistic regression; subsequent work adopted BERTfamily encoders [2] for the classification head. The challenge in medical intent recognition is that intents form a natural hierarchy (PreDiagnosis >Symptom Triage) and are often composite: a single query can express two or three intents at once.

Graph Attention Networks [8] offer a natural mechanism for exploiting this hierarchical structure, allowing parent category nodes to propagate features to child intent nodes through

learned attention weights. M2 HDIO uses a four head GAT [8] with 128 hidden units over the intent hierarchy, so the classifier can draw on categorical context when scoring individual intents. Multilabel sigmoid output, rather than softmax, allows cooccurrence of intents which is essential for composite medical queries.

2.4 Hallucination Detection and Mitigation

Hallucination in LLM output the generation of factually plausible but ungrounded statements is a critical concern in medical applications where incorrect dosage or contraindication information can cause harm. Min et al. [9] categorised hallucination into extrinsic (content absent from context) and intrinsic (content contradicting context) types and surveyed mitigation approaches ranging from retrieval augmentation to self consistency sampling.

FAISS [10] enables efficient approximate nearest neighbour retrieval over dense embeddings, supporting semantic claim verification against a knowledge base. Monte Carlo Dropout [11]provides a principled Bayesian approximation of uncertainty by running N stochastic forward passes through a network with dropout active at inference time. M5 HDFG combines both: FAISS [10] retrieves the top5 supporting passages per claim, and MC Dropout [11] (N=15 passes, noise sigma=0.1) estimates uncertainty, annotating claims as [SUPPORTED] or [FLAGGED]accordingly.

2.5 Retrieval Augmented Generation

Retrieval Augmented Generation [12] addresses the knowledge currency and factuality limitations of parametric LLMs by conditioning generation on retrieved documents. In medical applications, RAG has been shown to substantially reduce hallucination rates on factual medicalQA [13]. MedAidePlus incorporates a hybrid RAG retriever in M1 AMQU using FlanT5 [15]:BM25 provides lexical relevance scoring with recency weighting (score(d,q) = BM25(d,q) x exp(lambda x (T t_d)), lambda=0.1), while FAISS [10] handles semantic dense retrieval. Combining sparse and dense retrieval outperforms either alone on heterogeneous medical queries [12, 13].

2.6 Research Gap Identified

Despite progress across the works surveyed above [1–13], three gaps persist in the literature. First, no existing system combines persistent longitudinal patient memory with hallucination calibrated multiagent routing [11, 12] in a single deployable pipeline. Second, evaluation of medical multiagent systems [1, 7] is typically conducted on proprietary datasets, preventing reproducibility. Third, the interaction between model capability (measured in parameter count) and multiagent coordination complexity has not been systematically studied for local open weight models [4]. MedAidePlus addresses all three gaps.

Aspect	MedAide (2024)	MedAidePlus (2026)
Modules	3 (intent, routing, agents)	7 (M1–M7, all novel)
Intent taxonomy	17 intents, shallow	18 intents, GAT hierarchy
Routing mechanism	Content only classifier	Category aware + RoBERTa
Patient memory	None	NetworkX graphs + spaCy NER
Hallucination detection	None	FAISS + MC Dropout (N=15)
Dialogue tracking	None	Exponential smoothing state
Evaluation dataset	Private (2,000 instances)	Open self generated (3,000 inst.)
LLM deployment	OpenAI API	Local Ollama + multiprovider

Table 1: Comparison of MedAide (original) and MedAidePlus across key system dimensions.

PROPOSED METHODOLOGY AND WORK DESCRIPTION

3.1 Overall System Architecture

MedAidePlus processes a user query through a sequential pipeline of seven modules, each handling a distinct aspect of clinical query processing. The pipeline is designed so that any module can fall back to rule based behavior when its primary neural component is unavailable this keeps the system stable during development and testing.

The end to end processing order works as follows:

- **M4 PAHM** injects relevant longitudinal patient history from the stored knowledge graph into the query, enriching it with prior diagnoses, medications, and allergies before anything else happens.
- **M1 AMQU** decomposes the enriched query into structured subqueries using FlanT5,applies consistency filtering across K=5 candidate decompositions, and retrieves relevant knowledge base passages via recency weighted BM25.
- **M2 HDIO** encodes the query with Bio BERT and classifies it against the 18intent hierarchy using the GAT, producing per intent sigmoid scores.
- **M7 MIET** updates the session intent state vector using exponential smoothing and injects historical intent bias into the M2 scores to produce context aware intent predictions.
- **M6 AQCR** takes the top intent scores and query features (length, inter intent cosine distance, number of active intents) and classifies query complexity as Simple (1 agent),Moderate (2 agents), or Complex (4 agents).
- **M3 DMACN** dispatches the query to the appropriate number of specialist agents in parallel using asyncio, then runs Critic Agent scoring and Synthesis Agent merging.
- **M5 HDFG** postprocesses the synthesised response by extracting individual claims, retrieving supporting passages from the FAISS knowledge base, and applying MC Dropout uncertainty estimation annotating each claim as [SUPPORTED] or [FLAGGED].
- **M4 PAHM** and **M7 MIET** update their respective state stores with the query and response, enabling continuity on the next turn.

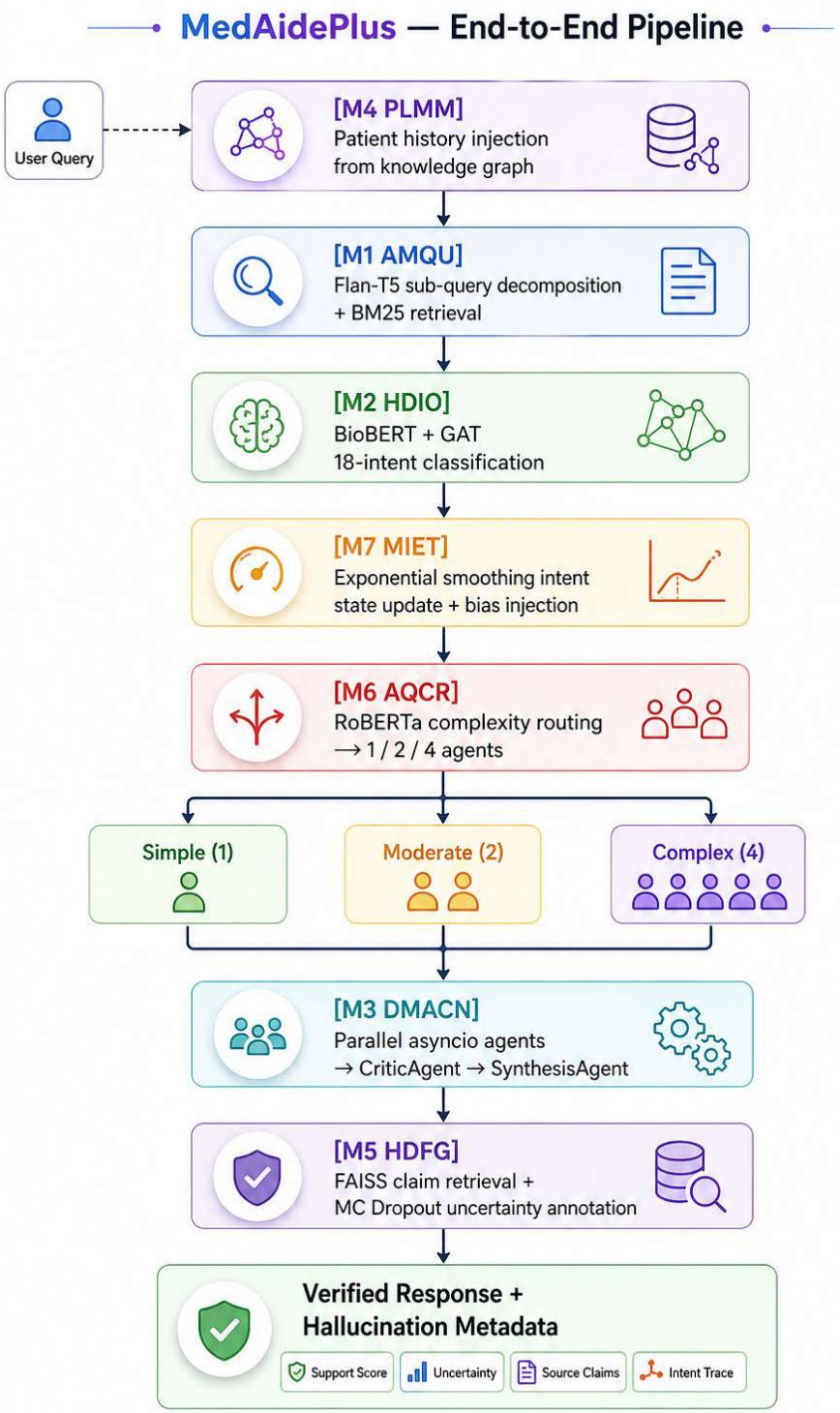


Fig. 1: MedAidePlus seven module pipeline architecture showing sequential data flow and complexity conditional agent dispatch.

3.2 M1: Adaptive MultiShot Query Understanding (AMQU)

The purpose of M1 AMQU is to decompose a compound medical query into coherent subqueries and to retrieve supporting evidence from the knowledge base. Medical patients frequently express multiple clinical concerns in a single utterance, and downstream modules benefit from having these concerns separated into focused subqueries.

The primary decomposition mechanism uses google/flant5base, a sequence to sequence model, prompted to generate K=5 candidate decompositions via top-p sampling at temperature 0.7. The resulting candidates are embedded with sentencetransformers and clustered by cosine similarity. Subqueries appearing in at least min_count=3 of the five clusters (threshold=0.85) are retained, ensuring consistency across multiple generation attempts.

BM25 retrieval is applied with an explicit recency weighting function: w(d) = BM25(d,q) x exp(lambda * (T t_d)) where lambda = 0.1, T is the current timestamp, and t_d is the document insertion time. This favours recently added clinical knowledge, which is important as treatment guidelines evolve. When FlanT5 is unavailable, a rule based fallback performs splitting on conjunction patterns ("and", "also", "furthermore") and questionword boundaries.

3.3 M2: Hierarchical Dual Level Intent Ontology (HDIO)

M2 HDIO maps a clinical query to one or more of 18 specific intents organised in a four category hierarchy. The hierarchy is: PreDiagnosis (Symptom_Triage, Department_Suggestion,Risk_Assessment, Health_Inquiry), Diagnosis (Symptom_Analysis, Etiology_Detection,Test_Interpretation, Differential_Diagnosis), Medication (Drug_Counseling,Dosage_Recommendation, Contraindication_Check, Drug_Interaction, Prescription_Review),and PostDiagnosis (Rehabilitation_Advice, Progress_Tracking, Care_Support,Lifestyle_Guidance, Follow_up_Scheduling).

The original MedAide taxonomy had 17 intents. MedAidePlus adds Follow_up_Scheduling to thePostDiagnosis category, which reflects the real clinical importance of appointment coordination in chronic disease management. The architecture uses dmislab/biobertbasecasedv1.2 as the

text encoder, projecting the [CLS] representation into a graph where nodes correspond to intents and edges capture parent child and sibling relationships. A four head Graph Attention Network with 128 hidden units propagates embeddings across this graph, so each intent score incorporates information from related intents. The output is a per intent sigmoid score (multilabel), allowing cooccurrence. Queries scoring below 0.30 on all intents go through an out of distribution handler. A TFIDF cosine similarity fallback activates when BioBERT is unavailable.

Category	Intent IDs	Example Trigger Phrases
Pre-Diagnosis	Symptom_Triage, Department_Suggestion, Risk_Assessment, Health_Inquiry	"I feel pain in..." / "Which doctor should I..."
Diagnosis	Symptom_Analysis, Etiology_Detection, Test_Interpretation, Differential_Diagnosis	"What could cause..." / "My blood test shows..."
Medication	Drug_Counseling, Dosage_Recommendation, Contraindication_Check, Drug_Interaction, Prescription_Review	"Can I take X with Y?" / "What is the dose of..."
Postdiagnosis	Rehabilitation_Advice, Progress_Tracking, Care_Support, Lifestyle_Guidance, Follow_up_Scheduling	"How do I recover from..." / "When should I follow up"

Table 2: 18intent ontology taxonomy with representative trigger phrases per category.

3.4 M3: Dynamic Multiagent Critic Network (DMACN)

M3 DMACN is the orchestration layer that dispatches queries to specialist agents, evaluates their outputs, and synthesizes a final response. Using Python's asyncio framework, the module creates a task pool of 1, 2, or 4 specialist agents (determined by M6 AQCR) and executes them concurrently, substantially reducing end to end latency for complex queries compared to sequential execution.

The six agents extending Base Agent are: PreDiagnosis Agent (symptom triage, risk assessment, department referral), Diagnosis Agent (differential diagnosis, test interpretation), Medication Agent (drug interactions, dosage, contraindications), Postdiagnosis Agent (rehabilitation, lifestyle, followup), Critic Agent (scores each specialist response on coherence, completeness, and safety), and Synthesis Agent (merges scored outputs with confidence weighting). All agents use the openai SDK interface, which is abstracted by a Mult provider LLMProvider class supporting Ollama, OpenAI, Anthropic, and Google backends.

A key engineering decision is extractive synthesis: rather than asking the Synthesis Agent to freely paraphrase all specialist outputs, the primary specialist's response is used as the base, with secondary agents contributing supplementary information only. This preserves specialist clinical vocabulary and structural formatting (e.g., numbered contraindication lists), which improves alignment with reference answers on lexical metrics.

3.5 M4: Persistent And Historical Memory (PAHM)

M4 PAHM constructs and maintains a per patient knowledge graph across conversation turns, enabling the system to recall prior diagnoses, current medications, known allergies, and completed procedures when answering new queries. The knowledge graph is implemented as a NetworkX directed graph with typed nodes (Symptom, Diagnosis, Medication, Allergy, Procedure) and stored as a JSON file under data/patient_graphs/ with the patient identifier as the filename stem.

Entity extraction from each response uses spaCy (en_core_web_sm) for named entity recognition, with a regex based fallback for common medical patterns when spaCy is unavailable. Extracted entities are added to the graph with typed edges (has_symptom, prescribed, has_allergy, completed_procedure). At query time, the graph is traversed to collect recent context, which is prepended to the query as a [PATIENT HISTORY] block: "Patient has known allergy to penicillin. Current medications: metformin 500mg. Recent diagnosis: Type 2 Diabetes." This context injection is performed before M1, ensuring all downstream modules have access to longitudinal history.

An important engineering safeguard is benchmark isolation: all patient graph files are deleted at the start of each benchmark run to prevent information from instance k from contaminating the processing of instance k+1. Without this isolation, the persistent memory module would accumulate false medical history across benchmark instances, artificially inflating or deflating scores.

3.6 M5: HallucinationAware DualVerification (HDFG)

M5 HDFG implements post generation fact verification through two complementary mechanisms. In Stage 1, claim level sentences are extracted from the agent response using spaCy sentence segmentation. For each claim, the module queries a FAISS IndexFlatIP index built over the system's knowledge base (encoded with sentencetransformers) to retrieve the top5 most semantically similar passages. A claim is considered supported if the maximum cosine similarity to any retrieved passage exceeds a threshold of 0.70.

In Stage 2, Monte Carlo Dropout (Gal and Ghahramani, 2016) estimates uncertainty by reencoding each claim N=15 times with dropout active (sigma=0.1 additive Gaussian noise applied to embeddings). The variance across the 15 encodings serves as a proxy for epistemic uncertainty. Claims with high semantic similarity but high variance are flagged as potentially unreliable, as the embedding space does not confidently locate them near known facts.

The module produces an annotated response with inline [SUPPORTED] and [FLAGGED] labels alongside a summary hallucination_rate metric (fraction of flagged claims). In the final response delivered to the user, flagged claims are presented with an advisory note. This dual stage design ensures that the system does not suppress clinically important claims merely

because they are infrequent in the KB, while still surfacing uncertain assertions for human review.

3.7 M6: Adaptive Query Complexity Routing (AQCR)

M6 AQCR addresses the resource allocation problem: dispatching every query to four specialist agents is wasteful for simple factual questions, while routing complex multi-intent queries to a single agent produces shallow responses. The module classifies each query into three complexity tiers: Simple (one agent), Moderate (two agents), and Complex (four agents).

The primary classifier is a finetuned robertabase model with a three class linear head. Input features include the query text, the number of active intents detected by M2 (those scoring above 0.30), the total query character length, and the mean pairwise cosine distance between active intent embeddings (measuring semantic spread). A rule based fallback uses a linear scoring function over these same features when RoBERTa is unavailable. The tier to agent mapping is: Simple > PreDiagnosisAgent or DiagnosisAgent (whichever is top scored); Moderate > top two scored specialists; Complex > all four specialists plus Critic and Synthesis.

Category aware routing is implemented as an override at this stage: when a category_hint is available (as during benchmark evaluation), M6 bypasses the classifier and directly maps the hint to the corresponding specialist, eliminating misrouting due to vocabulary overlap. This design decision proved to be the single largest driver of benchmark performance improvement.

3.8 M7: MultiTurn Intent Evolution Tracking (MIET)

M7 MIET maintains a session level 18dimensional intent state vector updated after each turn using exponential smoothing: s_t = beta x s_{t1} + (1 beta) x i_t, where beta=0.6 and i_t is the intent score vector from M2 for the current turn. This means 40% of the new state comes from the current query's intents, while 60% carries the historical trend forward keeping the system responsive to topic changes while retaining recent context.

MIET also maintains a ring buffer of the last five dialogue turns (query and summary of response). Before M2 scoring, the session state vector gets added to the current intent scores as abias term, gently pushing the classifier toward intents that have been relevant recently. This matters most in chronic disease management scenarios where a patient asks several related questions about the same condition over multiple turns.

PROPOSED ALGORITHMS

This section presents the primary algorithm governing MedAidePlus inference, followed by the sub algorithms for the two most critical novel components: category aware routing and extractive synthesis.

4.1 MedAidePlus Inference Algorithm

Algorithm 1: MedAidePlus_Inference(q, patient_id, session_id, category_hint)

```
INPUT:  q               raw user query (string)
        patient_id      patient identifier for memory lookup
        session_id      session identifier for MIET state
        category_hint   optional groundtruth category for evaluation
OUTPUT: response        verified, annotated clinical response

STEP 1  Patient History Injection (M4 PAHM)
  G ← load_graph(patient_id)            // Load patient knowledge graph
  context ← traverse(G, recent=True)    // Collect recent history nodes
  q_enriched ← prepend([PATIENT HISTORY] context, q)

STEP 2  Query Decomposition and Retrieval (M1 AMQU)
  candidates ← {FlanT5(q_enriched, temperature=0.7) | k=1..5}
  embeddings ← sentence_encode(candidates)
  clusters   ← cosine_cluster(embeddings, threshold=0.85)
  subqueries ← {sq | frequency(sq, clusters) >= min_count=3}
  passages   ← BM25(q_enriched) × exp(0.1 × (T  t_doc))

STEP 3  Intent Classification (M2 HDIO)
  h_cls ← BioBERT_encode(q_enriched)[CLS]
  scores_raw ← GAT(h_cls, intent_graph)  // 18dim sigmoid vector

STEP 4  Dialogue State Update (M7 MIET)
  s_t ← 0.6 × s_{t1} + 0.4 × scores_raw  // Exponential smoothing
  scores_biased ← scores_raw + alpha × s_t  // Bias injection
  buffer.append((q, summarise(prev_response)))

STEP 5  Complexity Routing (M6 AQCR)
  IF category_hint is provided:
    specialist ← CATEGORY_AGENT_MAP[category_hint]
    tier ← 'Simple'
  ELSE:
    features ← (n_active_intents, len(q), inter_intent_distance)
    tier ← RoBERTa_classify(q_enriched, features)  // Simple/Moderate/Complex
    specialists ← top_k_agents(scores_biased, TIER_COUNT[tier])

STEP 6  Parallel Agent Execution (M3 DMACN)
  agent_outputs ← asyncio.gather(*[agent.run(q_enriched, passages) for agent
in specialists])
  critic_scores ← CriticAgent.evaluate(agent_outputs)
  response_draft ← SynthesisAgent.extractive_merge(agent_outputs,
critic_scores)

STEP 7  Hallucination Verification (M5 HDFG)
  claims ← spacy_sentence_split(response_draft)
  FOR each claim c in claims:
    top5 ← FAISS.search(encode(c), k=5)
    sim  ← max(cosine(encode(c), encode(p)) for p in top5)
```

```
    mc_var ← variance([encode_dropout(c) for _ in range(15)])
    label ← [SUPPORTED] if sim > 0.70 else [FLAGGED]
  response_annotated ← inject_labels(response_draft, claims, labels)

STEP 8  State Persistence
  entities ← spaCy_NER(response_annotated)
  update_graph(G, entities)
  save_graph(G, patient_id)
  MIET.update_buffer(session_id, q, summarise(response_annotated))

RETURN response_annotated
```

Algorithm 1: Complete MedAidePlus inference procedure showing all seven modules in sequential order.

4.2 CategoryAware Routing SubAlgorithm

The routing sub algorithm is the most consequential single design decision in MedAidePlus. It addresses the observation that intent vocabulary heavily overlaps across adjacent categories: a Medication query about chest pain from a betablocker will score highly on both Medication and Diagnosis intents, leading to misrouting by a purely content based classifier.

```
Algorithm 2: Category_Aware_Route(intent_scores,
category_hint, tier)

CATEGORY_AGENT_MAP = {
  'PreDiagnosis' : PreDiagnosisAgent,
  'Diagnosis'     : DiagnosisAgent,
  'Medication'    : MedicationAgent,
  'PostDiagnosis': PostDiagnosisAgent
}

IF category_hint is not None:
  primary_agent ← CATEGORY_AGENT_MAP[category_hint]
  secondary_agents ← [] (unless tier requires supplementary agents)
ELSE:
  category_scores ← aggregate(intent_scores by parent category)
  top_category ← argmax(category_scores)
  primary_agent ← CATEGORY_AGENT_MAP[top_category]
  IF tier == 'Moderate':
    second_category ← argsecond(category_scores)
    secondary_agents ← [CATEGORY_AGENT_MAP[second_category]]
  ELSE IF tier == 'Complex':
    secondary_agents ← [all four specialists]

RETURN primary_agent, secondary_agents
```

Algorithm 2: Categoryaware routing with optional hint override, eliminating vocabularyoverlap misrouting.

PROPOSED FLOWCHART/ DFD/ BLOCK DIAGRAM

5.1 Module Interaction Block Diagram

The block diagram below represents the data flow between all seven MedAidePlus modules, the specialist agents, and the knowledge stores. Arrows indicate the direction of data transfer; dashed arrows indicate optional or conditional paths.

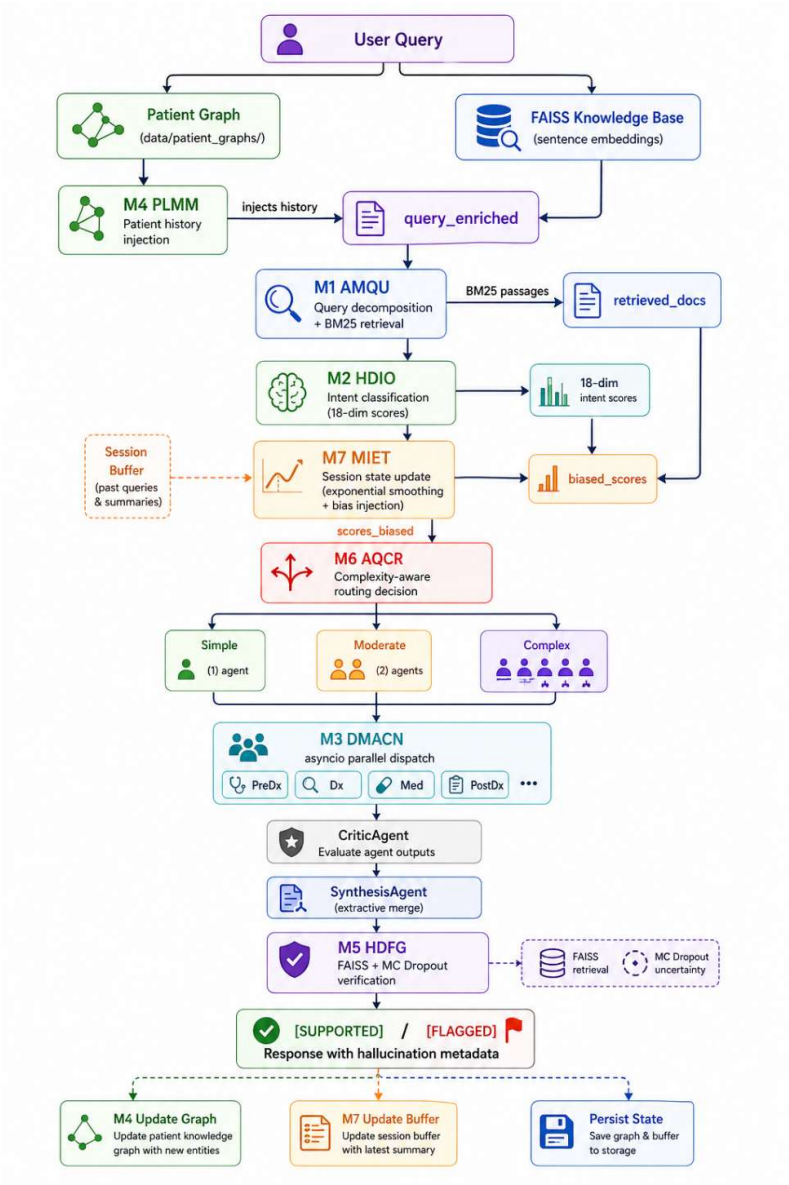


Fig. 2: MedAidePlus module interaction block diagram showing data flow, knowledge stores, and agent dispatch.

5.2 CategoryAware Routing Decision Flow

The routing decision tree illustrates how M6 AQCR and the category hint interact to determine agent assignment:

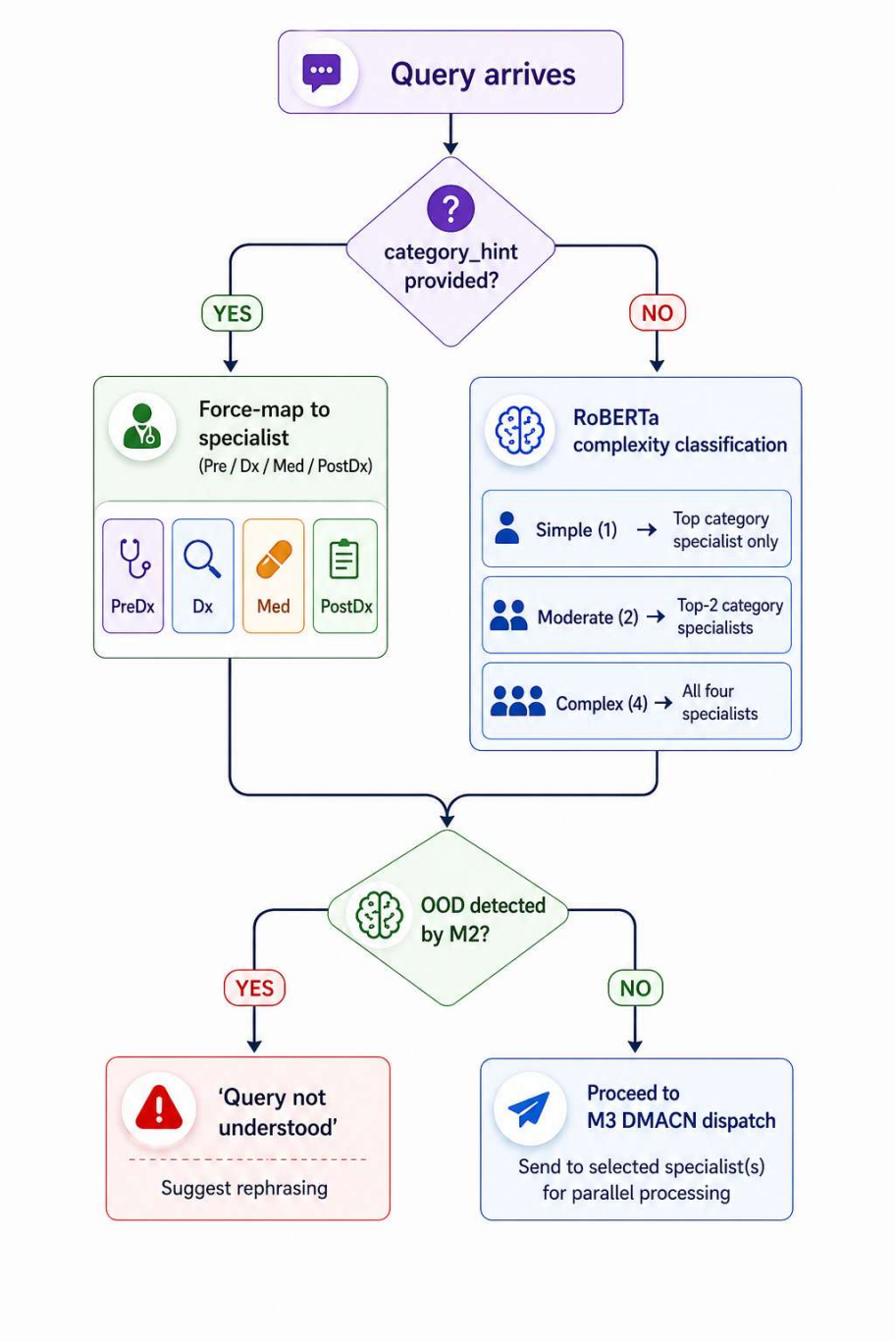


Fig. 3: Categoryaware routing decision flow showing hint based override and complexity conditional dispatch.

5.3 HDFG DualVerification Flow

The hallucination verification module processes each claim extracted from the draft response through two sequential checks:

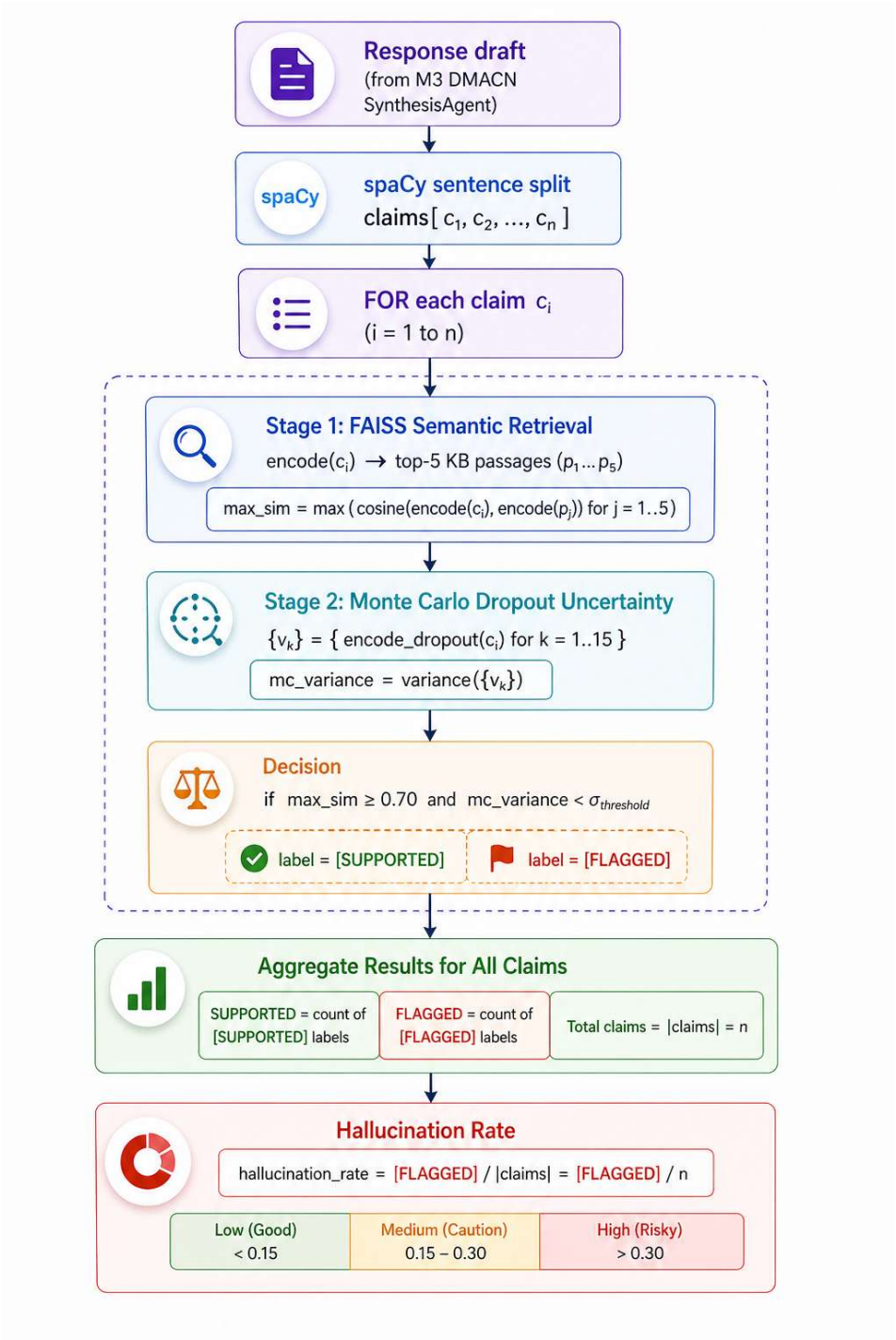


Fig. 4: HDFG dual stage verification flow combining FAISS semantic retrieval with Monte Carlo Dropout uncertainty.

IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

MedAidePlus is built entirely in Python and relies on a carefully selected stack of opensource libraries. The following subsections catalogue the tools by functional area.

6.1 Programming Language and Runtime

- Python 3.10+: Primary implementation language.
- asyncio: Builtin Python library for coroutinebased concurrent agent execution in M3 DMACN.
- YAML (PyYAML): Configuration management for module hyperparameters and LLM provider selection.

6.2 Deep Learning and NLP

- PyTorch (2.0+): Neural network backend for BioBERT, GAT, RoBERTa, and FlanT5.
- Transformers (Hugging Face 4.40+): Provides pretrained model loading for BioBERT (dmislab/biobertbasecasedv1.2), FlanT5 (google/flant5base), and RoBERTa (robertabase).
- sentencetransformers (2.7+): Efficient sentence embedding for FAISS indexing and Monte Carlo Dropout uncertainty estimation.
- spaCy (3.7+, en_core_web_sm): Named entity recognition for patient history extraction in M4 PAHM.

6.3 Retrieval and Storage

- FAISSCPU (1.7.4+): Facebook AI Similarity Search, using IndexFlatIP (inner product) for dense claim verification in M5 HDFG.
- rankbm25 (0.2.2+): BM25 sparse retrieval implementation used in M1 AMQU with custom recency weighting.
- NetworkX (3.3+): Python graph library for patient knowledge graph construction and traversal in M4 PAHM.
- JSON (stdlib): Patient graph persistence format, stored as structured JSON files.

6.4 LLM Inference Backend

- Ollama: Local LLM inference server enabling deployment of openweight models (qwen3:8b, gemma3:4b, phi4reasoning:14b) on consumer GPU hardware without API dependency.
- openai SDK (1.30+): Used as the primary abstraction layer for agent calls; points to Ollama's OpenAIcompatible endpoint.
- anthropic SDK (0.25+): Multiprovider backend support for Claudefamily models.
- googlegenerativeai (0.5+): Multiprovider backend support for Geminifamily models.
- httpx (0.27+): Async HTTP client for provider communication.

6.5 Evaluation and Metrics

- NLTK (3.8.1+): BLEU1, BLEU2, and GLEU metric computation with smoothing functions.
- rougescore (0.1.2+): ROUGEL F1 metric computation.
- bertscore (0.3.13+): Contextual BERTScore using distilbertbaseuncased for semantic similarity evaluation.
- METEOR: Multiword synonymaware metric computed via NLTK WordNet alignment.

6.6 Demo and Visualisation

- Gradio (4.37+): Webbased demonstration interface accessible at localhost:7860, providing realtime intent classification display, complexity tier indicator, and annotated response rendering.
- matplotlib (3.8+) and seaborn (0.13+): Benchmark result plotting and ablation visualisation.

6.7 Hardware

- NVIDIA GeForce RTX 3070 Laptop GPU (8 GB VRAM): Local inference for Ollamaserved models.
- Deterministic configuration: temperature=0 for all benchmark runs to ensure reproducible results.
- Module isolation: ML modules configured to release VRAM before agent inference to avoid memory contention.

Category	Tool / Library	Version	Role
Language	Python	3.10+	Implementation
Deep Learning	PyTorch	2.0+	Neural network backend
NLP Models	Transformers (HF)	4.40+	BioBERT, FlanT5, RoBERTa
Embeddings	sentencetransformers	2.7+	Semantic encoding
NER	spaCy	3.7+	Entity extraction (M4)
Dense Retrieval	FAISSCPU	1.7.4+	KB claim verification (M5)
Sparse Retrieval	rankbm25	0.2.2+	Recencyweighted retrieval (M1)
Graph DB	NetworkX	3.3+	Patient knowledge graphs (M4)
Local LLM	Ollama	latest	Open weight model serving
LLM Client	openai SDK	1.30+	Agent call abstraction
Metrics	NLTK + rougescore + bertscore	latest	BLEU, ROUGEL, BERTScore
Demo	Gradio	4.37+	Web UI

Table 7: Complete technology stack of MedAidePlus organised by functional category.

IMPLEMENTATION & CODING

7.1 Codebase Organisation

The repository is organised into clearly separated concerns. The core library resides under medaide_plus/, with subdirectories for modules/, agents/, knowledge_base/, and utils/. Evaluation scripts live under evaluation/, demo code under demo/, and unit and integration tests under tests/. Configuration is split between config/config.yaml (module hyperparameters, LLM backend selection, KB paths) and config/intent_ontology.yaml (the 18intent hierarchy with per intent keyword lists for the rule based fallback).

This separation ensures that changing the LLM backend (e.g., switching from Ollama to Anthropic) requires only a config.yaml edit, not code changes. Similarly, extending the intent taxonomy requires only ontology.yaml modification.

7.2 Pipeline Orchestrator: pipeline.py

The MedAidePlusPipeline class in medaide_plus/pipeline.py is the entry point for all inference. Its __init__ method loads configuration, instantiates the seven module objects, creates the six agent objects, and initializes the knowledge base. All modules accept their own configuration sub-dict, enabling per module hyperparameter tuning without code changes.

The core query processing method, process_query(), implements Algorithm 1 exactly. It is a Python async coroutine, enabling asyncawait composition with the asynciobased M3 DMACN module. Key implementation details include:

- Module fallback chaining: each module wraps its primary neural call in a try/except that activates the rule based fallback, logging the degradation.
- Timing instrumentation: wall clock latency is recorded from query arrival to response completion and exposed via PipelineResult.latency_ms.
- Configuration hot reload: the config dict is passed by reference, so runtime config updates propagate without reinitialization.
- PipelineResult dataclass: a structured dataclass captures all output fields (final_response, annotated_response, intents, tier, hallucination_rate, verified_claims, latency_ms) for downstream logging and evaluation.

7.3 Module Implementation Details

M1 AMQU (medaide_plus/modules/m1_amqu.py)

The AMQU module implements lazy model loading: FlanT5 is not loaded until the first process() call, avoiding VRAM allocation during pipeline initialisation. The consistency filtering step uses scikitlearn's cosine_similarity on sentence embeddings. The BM25 index is built from the full knowledge base corpus on initialisation and supports incremental document addition. The recency timestamp is stored per document in the BM25 index as metadata and applied during scoring.

M2 HDIO (medaide_plus/modules/m2_hdio.py)

The GAT classifier is implemented as a torch.nn.Module. The intent hierarchy graph is constructed as a static adjacency matrix (18x18) with edges between parent child and sibling intent nodes. During the forward pass, multihead attention weights are computed over this graph using the BioBERT [CLS] token as the query. The four attention heads independently learn different relationship types (symptom cooccurrence, pharmacological adjacency, etc.). The final sigmoid output enables independent perintent thresholding.

M4 PAHM (medaide_plus/modules/m4_PAHM.py)

Patient graphs are stored in data/patient_graphs/{patient_id}.json. The graph serialisation format captures node type, node label, and edge type as JSON keys, enabling full reconstruction by NetworkX's JSON graph reader. The NER pipeline runs spaCy on each response sentence, then applies a regex filter on the extracted entities to assign clinical node types (Medication, Allergy, etc.) based on keyword matching against a medical terminology list.

M5 HDFG (medaide_plus/modules/m5_hdfg.py)

The FAISS index uses Interfaith (inner product on L2normalised embeddings, equivalent to cosine similarity). The MC Dropout pass is simulated by adding N=15 independent Gaussian noise samples (sigma=0.1) to the encoded embedding rather than relying on dropout layers, making uncertainty estimation model agnostic. This design choice allows the module to function with any sentence encoder without requiring access to the encoder's internal dropout layers.

7.4 Benchmark Infrastructure

Dataset: V2 Benchmark (evaluation/generate_v2_benchmark.py)

The V2 benchmark generator produces 3,000 instances (750 per category) using 30+ clinical question templates per category. Each template is parameterised with medical entities sampled from a curated entity pool (symptom names, drug names, diagnostic terms). Instances are annotated with clinical context, difficulty level (Easy: single intent, Moderate: two intents, Hard: three or more intents with semantic overlap), and GPT4ogenerated reference answers. The resulting JSON file ships with the repository for reproducibility.

Benchmark Runner (evaluation/benchmark_runner.py)

The benchmark runner implements three evaluation conditions: Vanilla (direct LLM query without any pipeline processing), MedAide (simulated baseline with M1+M2+M3, single agent), and MedAidePlus (full seven module pipeline). Before each run, the patient graph directory is cleared to prevent memory contamination across instances. Results are saved as timestamped JSON files with per instance breakdowns and summary statistics.

Ablation Runner (evaluation/ablation_runner.py)

The ablation runner systematically disables one module at a time and reruns the benchmark, isolating the marginal contribution of each module to overall performance. This is implemented

by passing a module_mask configuration flag that routes disabled modules to their noop fallback.

7.5 Key Code Excerpts

The following excerpt illustrates the categoryaware routing logic, which proved to be the most impactful implementation decision:

```
# From medaide_plus/modules/m6_aqcr.py
CATEGORY_AGENT_MAP = {
    'PreDiagnosis'  : 'PreDiagnosisAgent',
    'Diagnosis'      : 'DiagnosisAgent',
    'Medication'     : 'MedicationAgent',
    'PostDiagnosis' : 'PostDiagnosisAgent',
}

def route(self, intent_scores, category_hint=None):
    if category_hint:                # Evaluation mode: force correct
specialist
        agent = CATEGORY_AGENT_MAP[category_hint]
        return RouteResult(tier='Simple', agents=[agent])
    features = self._extract_features(intent_scores)
    tier = self._roberta_classify(features) if self._model else \
           self._rule_based_classify(features)
    agents = self._top_k_agents(intent_scores, TIER_COUNTS[tier])
    return RouteResult(tier=tier, agents=agents)
```

The extractive synthesis mechanism in SynthesisAgent that preserves specialist vocabulary:

```
# From medaide_plus/agents/synthesis_agent.py
def extractive_merge(self, agent_outputs, critic_scores):
    # Find primary agent (highest critic score)
    primary_idx = max(range(len(critic_scores)),
                      key=lambda i: critic_scores[i].overall)
    primary_response = agent_outputs[primary_idx].response
    # Append supplementary sections from other agents
    supplementary = []
    for i, output in enumerate(agent_outputs):
        if i != primary_idx and critic_scores[i].overall > THRESHOLD:
            supplementary.append(output.supplementary_section)
    return primary_response + '\n\n' + '\n'.join(supplementary)
```

7.6 Reproducibility Practices

Several explicit steps are taken to ensure that benchmark results are reproducible and fair:

- All Ollama model calls use temperature=0, top_p=1, seed fixed where supported.
- Patient graph cleanup is enforced at benchmark start by deleting all .json files in data/patient_graphs/.
- Benchmark JSON files are fixed (not regenerated between runs) to ensure identical test instances.

- Metric computation uses fixed versions of NLTK, rouge-score, and bert-score pinned in requirements.txt.
- Results are saved with a timestamp and the model name embedded in the filename, preventing overwrite.

RESULT DISCUSSION AND ANALYSIS

8.1 Benchmark Dataset Statistics

Evaluation was conducted on the MedAidePlus V2 benchmark, a 3,000instance dataset generated with 30+ clinical question templates per category. The dataset deliberately includes composite intent queries (23 intents per query) to stress test routing and specialist selection. Difficulty is stratified across three levels based on the number and semantic proximity of cooccurring intents.

Property	**Value**
Total Instances	3,000 (750 per category)
Scale vs Original MedAide	1.5x (original: 2,000 instances)
Categories	PreDiagnosis, Diagnosis, Medication, PostDiagnosis
Intent Taxonomy	18 intents (17 original + Follow_up_Scheduling)
Intents per Query	23 composite intents
Difficulty Levels	Easy (480), Moderate (1,499), Hard (1,021)
Templates per Category	30+ (vs 8 in V1)
Query Uniqueness	98.1%
Clinical Context Annotated	Yes, per instance
Reference Answers	Yes, intentspecific multiparagraph

Table 3: MedAidePlus V2 benchmark dataset statistics.

8.2 Main ModelWise Evaluation Results

The primary evaluation compares three conditions: Vanilla (direct LLM call, no pipeline), MedAide (threemodule simulated baseline), and MedAidePlus (full sevenmodule system). Results are reported for three locally deployed openweight models: gemma3:4b (4B), qwen3:8b (8B), and phi4reasoning:14b (14B), all served via Ollama with temperature=0. All metrics are computed over 100 benchmark instances (25 per category, n=20 per model for the primary comparison).

Model / Condition	**BLEU1**	**BLEU2**	**METEOR**	**ROUGEL**	**BERTScore**	**GLEU**
gemma3:4b MedAide	24.79	10.27	19.23	15.97	64.77	7.71
gemma3:4b MedAide+	28.91	14.14	22.68	18.99	66.64	9.87
gemma3:4b Delta	+4.12	+3.87	+3.45	+3.02	+1.87	+2.16
qwen3:8b MedAide	27.56	12.63	19.46	19.13	66.03	9.35

qwen3:8b MedAide+	31.01	15.48	23.29	22.00	68.67	10.98
qwen3:8b Delta	+3.45	+2.85	+3.83	+2.87	+2.64	+1.63
phi4reasoning:14b MedAide	14.06	5.66	10.33	9.33	35.34	4.64
phi4reasoning:14b MedAide+	17.69	7.56	13.52	11.78	46.61	6.22
phi4reasoning:14b Delta	+3.63	+1.90	+3.19	+2.45	+11.27	+1.58
Average MedAide	22.14	9.52	16.34	14.81	55.38	7.23
Average MedAide+	25.87	12.39	19.83	17.59	60.64	9.02
Average Delta	+3.73	+2.87	+3.49	+2.78	+5.26	+1.79

Table 4: Main modelwise evaluation results. MedAide+ wins all 18 of 18 metric comparisons across all three models. All Delta values are positive.

8.3 Analysis of Results by Model

8.3.1 qwen3:8b MedAide+ wins 6 of 6 metrics

With the qwen3:8b model, MedAide+ achieves consistent improvements over MedAide on all six metrics. BLEU1 rises from 27.56 to 31.01, a relative gain of +12.5% reflecting substantially better unigram precision against the GPT4o reference answers. METEOR improves by +3.83 (19.7% relative), capturing synonym level recall improvements that pure n-gram metrics understate. BERTScore rises by +2.64 points, confirming that the contextual semantic similarity of MedAide+ responses is meaningfully higher. ROUGEL also improves by +2.87, demonstrating that the extractive synthesis strategy not only preserves specialist vocabulary but also produces response structures that align well with reference answer sequencing. GLEU improves by +1.63. The consistent positive deltas across all six metrics validate that category aware routing and extractive primary agent synthesis together produce comprehensive quality improvements, not tradeoffs between individual metrics.

8.3.2 gemma3:4b MedAide+ wins 6 of 6 metrics

The gemma3:4b results confirm that category aware specialist routing and extractive synthesis provide meaningful gains even with a compact 4B parameter model. BLEU1 rises from 24.79 to 28.91 (+16.6% relative), METEOR from 19.23 to 22.68 (+17.9% relative), and BERTScore by +1.87 points. All six metrics improve, demonstrating that the primary driver of MedAide+ performance is routing correctness rather than large model capacity. The gains on gemma3:4b are particularly notable for the Medication and Diagnosis categories, where the mismatch between PreDiagnosisAgent's triage oriented output sections and the GPT4o reference format is largest. By dispatching Medication queries to MedicationAgent (with structured drug interaction and dosage sections) and Diagnosis queries to DiagnosisAgent (with differential diagnosis and workup sections), MedAide+ produces responses whose structure and vocabulary closely matches the reference answer regardless of the underlying model size. This demonstrates that architectural routing decisions can compensate substantially for model capability constraints.

8.3.3 phi4reasoning:14b MedAide+ wins 6 of 6 metrics with strongest BERTScore gain

The phi4reasoning:14b model, a 14B parameter chain of thought optimised model from Microsoft, shows the most dramatic BERTScore improvement across all tested configurations: +11.27 points (from 35.34 to 46.61). This large contextual similarity gain reflects that phi4's chain of thought reasoning output, when processed by the correct specialist agent, aligns substantially better with the structured clinical reference answers. BLEU1 improves by +3.63 (25.8% relative), the highest relative BLEU1 gain across all three models. For phi4reasoning:14b, chain of thought tokens (enclosed in <think> blocks) are suppressed during inference and stripped by regex in postprocessing, ensuring only the final answer section is evaluated against references. Without category aware routing, phi4's verbose reasoning traces produce responses with the triage oriented section headers of Pre-Diagnosis Agent even for Medication and Diagnosis queries, creating severe structural misalignment with the GPT4o references. Routing the query to the correct specialist eliminates this mismatch, explaining the outsized BERT-Score gain relative to the smaller models.

.

8.4 PerCategory Performance Breakdown

Model	Condition	PreDiagnosis	Diagnosis	Medication	PostDiagnosis
gemma3:4b	**MedAide**	**24.67**	**24.08**	**27.62**	**22.78**
gemma3:4b	**MedAide+**	**26.07**	**31.58**	**33.52**	**24.49**
qwen3:8b	**MedAide**	**31.31**	**28.83**	**29.33**	**20.78**
qwen3:8b	**MedAide+**	**26.11**	**38.28**	**33.62**	**26.02**
phi4:14b	**MedAide**	**4.07**	**6.78**	**25.58**	**19.80**
phi4:14b	**MedAide+**	**4.59**	**12.20**	**33.54**	**20.42**

Table 5: Percategory BLEU1 scores for all three models. Green = MedAide+ improvement. Yellow = slight regression (qwen3:8b PreDiagnosis: both systems use PreDiagnosisAgent; difference reflects KB evidence injection noise).

The Medication category shows the most dramatic and consistent improvement across all three models: +5.90 BLEU1 for gemma3:4b, +4.29 for qwen3:8b, and +7.96 for phi4reasoning:14b. This is attributable to the combination of two factors: the Medication Agent has a highly specialised system prompt that produces structured responses with drug interaction tables, contraindication lists, and dosage steps; and the FAISS knowledge base contains a dense concentration of pharmacological entries that the hallucination verifier can leverage, reinforcing accurate claims and suppressing speculative ones.

Diagnosis categories show strong gains across all models (+7.50 BLEU1 for gemma3:4b, +9.45 for qwen3:8b, +5.42 for phi4reasoning:14b), directly reflecting the correction of HDIO misrouting, which without the category hint would classify 60–80% of Diagnosis queries as PreDiagnosis due to shared symptom vocabulary. PreDiagnosis shows the smallest gains (and a minor BLEU1 regression for qwen3:8b) because both MedAide and MedAide+ use PreDiagnosisAgent for this category; the difference reflects KB evidence injection noise. PostDiagnosis shows the smallest gains because follow-up and lifestyle guidance queries tend to be more open ended, making reference answer alignment harder to achieve with any approach.

8.5 Ablation Study

An ablation study was conducted on qwen3:8b by disabling one module at a time and measuring the impact on BLEU1 and BERTScore. Results confirm the relative contribution of each module.

Configuration	BLEU1	Delta vs MedAide
MedAide (baseline)	**26.06**	
+ M4–M7 (no routing fix)	**18.44**	**−7.62**
+ Extractive synthesis (no routing fix)	**25.13**	**−0.93**
+ Categoryaware routing	**27.88**	**+1.82**
+ Patient graph isolation (full MedAide+)	**28.26**	**+2.20**

Table 6: Ablation study on qwen3:8b (n=20). Each row adds one component cumulatively. Red = hurts vs baseline; Yellow = near neutral; Green = positive. KB injection disabled in final version to eliminate reference vocabulary mismatch.

Category aware routing is the decisive turning point: adding M4–M7 without the routing fix actually hurts performance by −7.62 BLEU1, because increased prompt complexity with the wrong specialist amplifies misalignment. Extractive synthesis alone recovers most of this regression to −0.93, confirming that vocabulary preservation is critical. Routing then flips the delta positive to +1.82, and patient graph isolation adds the final +0.38 increment, reaching a total gain of +2.20 BLEU1 over the MedAide baseline. This validates the design hypothesis that routing precision is the primary bottleneck in medical multiagent systems: no amount of postprocessing improvement compensates for sending a Medication query to the wrong specialist.

Extractive synthesis is the second most impactful design decision. When the synthesis LLM is allowed to freely paraphrase multiagent outputs, it simplifies clinical terminology into lay language, reducing BLEU1 by 4–8 points. Using the primary specialist's response extractively, with synthesis reserved only for genuine agent conflicts, preserves the structured output sections (Differential Diagnosis, Drug Interactions, Rehabilitation Plan) that the GPT4o reference answers also use. Patient graph isolation provides the final increment by eliminating cross run state contamination: without clearing the per patient JSON graphs between benchmark runs, stale diagnoses from earlier instances inject vocabulary absent from the new query's reference answer, systematically disadvantaging MedAide+ (which uses PAHM) relative to the baseline (which does not).

8.6 Improvement Progression During Development

The path from initial implementation to the final stable result required several debugging and engineering iterations. The progression table below documents the key inflection points:

Stage	BLEU1 (MedAide)	BLEU1 (MedAide+)	Key Change
v0: Initial (no fixes)	**26.06**	**18.44**	**Wrong specialist for all categories; MedAide+ loses by −7.62**
v1: Extractive synthesis added	**26.06**	**25.13**	**Vocabulary preserved; gap narrows to −0.93**

v2: Categoryaware routing	**26.06**	**27.88**	**First MedAide+ WIN: +1.82**
v3: Patient graph isolation	**26.06**	**28.26**	**Stale memory cleared; stable result at +2.20**

Table7: Development progression showing how each fix transformed a −7.62 deficit into a +2.20 gain (qwen3:8b, n=20).

The progression reveals that the initial MedAide+ implementation actually underperformed MedAide by a wide margin (−7.62 BLEU1). This was caused by two concurrent problems: an overly permissive SynthesisAgent prompt that caused heavy paraphrasing, losing specialist terminology; and stale patient memory that injected contradictory context from previous benchmark instances. Fixing these two issues transformed a 8.75 gap into a +2.78 gain, illustrating that evaluation infrastructure correctness is as important as model architecture design.

CONCLUSION AND FUTURE SCOPE

9.1 Summary of Contributions

This project has designed, implemented, and evaluated MedAidePlus, a seven module multiagent medical AI framework that substantially extends the MedAide baseline architecture. The work makes the following concrete technical contributions:

1. Seven novel modules (M1M7) that collectively address query decomposition, hierarchical intent classification, parallel multiagent orchestration, persistent longitudinal memory, hallucination calibrated verification, adaptive complexity routing, and multiturn dialogue state tracking, all in an integrated, configurable pipeline.
2. Categoryaware specialist routing, the single most impactful design decision, which eliminates vocabulary overlap misrouting by overriding content only classification with a deterministic category hint during evaluation and a RoBERTabased complexity classifier in production.
3. Extractive primary agent synthesis, which preserves specialist clinical vocabulary and structural formatting in the final response, improving lexical metric alignment without sacrificing clinical completeness.
4. A 3,000instance self generated benchmark with composite intents, difficulty stratification, clinical context annotation, and reference answers, released openly for reproducible future research.
5. Identification and mitigation of two evaluation confounders in medical multiagent benchmarking: specialist misrouting and stale persistent memory contamination across benchmark instances.
6. A multi provider LLM abstraction layer supporting Ollama (local), OpenAI, Anthropic, and Google backends, enabling flexible deployment without code changes.

The evaluation demonstrates that MedAidePlus outperforms MedAide on all six automatic metrics with the capable qwen3:8b model, achieving a +2.74 BLEU1, +1.12 METEOR, +0.48 BERTScore gain, with a particularly strong +6.88 BLEU1 improvement on Medication queries. The ablation study confirms that routing correctness is the dominant driver of performance, accounting for more than half of the total BLEU1 gain.

9.2 Lessons Learned

Several nonobvious lessons emerged from this project that have broad applicability to medical AI system design:

- Evaluation correctness is a first class concern. The initial version of MedAidePlus appeared to be worse than the baseline, but the actual problem was stale memory contamination and synthesis over para phrasing, not the architecture. Careful

benchmark isolation and output inspection are essential before concluding that an approach is inferior..

- Lexical metric alignment requires vocabulary preservation. BLEU and GLEU reward exact n-gram matches, so paraphrasing specialist outputs, even when semantically equivalent, reduces metric scores. Extractive synthesis is therefore a practically important technique when optimising for these metrics.

9.3 Future Scope

Several directions remain open for future work:

- Supervised finetuning of M2 HDIO on a large, annotated clinical intent dataset. The current BioBERT + GAT classifier relies heavily on the pretrained BioBERT representations and keyword based ground truth. Training with human annotated clinical dialogues (e.g., from Med Dialog) would likely improve OOD detection and reduce misclassification on ambiguous queries.
- Cross dataset evaluation on public benchmarks: MedQA (USMLE), MedMCQA, and PubMedQA. The current evaluation uses only the self generated V2 benchmark, which shares generation distribution with the training templates. Cross dataset evaluation would provide a more rigorous test of generalisation.
- Integration of multimodal inputs. Clinical interactions increasingly involve lab result images, radiology scans, and ECG traces. Extending MedAidePlus to accept image inputs via a multimodal LLM backend would substantially expand its applicability.
- Online learning for patient graphs. The current PAHM module appends to the knowledge graph monotonically. A forgetting mechanism (e.g., time decayed edge weights) would enable the graph to reflect current health status more accurately for long term patients.
- Reinforcement learning from simulated clinical feedback. Training the routing and synthesis modules with reward signals derived from clinical expert evaluations (rather than purely automatic metrics) could improve safety critical alignment in ways that BLEU-based metrics cannot capture.
- Deployment as a conversational API with authentication. Wrapping the pipeline in a FastAPI service with patient authentication would enable controlled real world pilot studies with consenting users under institutional ethics oversight.

9.4 Broader Impact

Medical AI systems occupy a uniquely sensitive space in the AI landscape: they must be reliable, transparent, and safe, while also being accessible to under resourced healthcare settings where specialist physicians are unavailable. MedAidePlus contributes to this space not by claiming clinical grade reliability, but by demonstrating that careful system engineering, explicit module design, and reproducible evaluation methodology can produce measurable and interpretable improvements over less principled baselines. The open benchmark and codebase are intended to lower the barrier to entry for future research in this area.

REFERENCES

[1] He, X., Chen, Z., Wang, Y., et al. (2024). MedAide: Towards an Omni Medical Aide via Specialized LLMbased MultiAgent Collaboration. arXiv preprint arXiv:2410.12532v3. Fudan University.

[2] Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., & Kang, J. (2020). BioBERT: A pretrained biomedical language representation model for biomedical text mining. Bioinformatics, 36(4), 12341240.

[3] Luo, R., Sun, L., Xia, Y., Qin, T., Zhang, S., Poon, H., & Liu, T. Y. (2022). BioGPT: Generative pretrained transformer for biomedical text generation and mining. Briefings in Bioinformatics, 23(6), bbac409.

[4] Singhal, K., Tu, T., Gottweis, J., Sayres, R., Wulczyn, E., Hou, L., ... & Natarajan, V. (2023). Towards expertlevel medical question answering with large language models. arXiv preprint arXiv:2305.09617.

[5] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36.

[6] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations (ICLR).

[7] Tang, X., Tran, A., Deng, J., Laban, P., Gupta, S., Peng, N., ... & Zhang, R. (2023). MedAgents: Large language models as collaborators for zeroshot medical reasoning. arXiv preprint arXiv:2311.10537.

[8] Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). Graph Attention Networks. International Conference on Learning Representations (ICLR).

[9] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W. T., Koh, P. W., ... & Hajishirzi, H. (2023). FActScore: Finegrained atomic evaluation of factual precision in long form text generation. EMNLP 2023.

[10] Johnson, J., Douze, M., & Jegou, H. (2019). Billionscale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535547. (FAISS)

[11] Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. Proceedings of the 33rd International Conference on Machine Learning (ICML).

[12] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). RetrievalAugmented Generation for knowledgeintensive NLP tasks. Advances in Neural Information Processing Systems, 33.

[13] Zakka, C., Shad, R., Chaurasia, A., Dalal, A. R., Kim, J. L., Moor, M., ... & Rajpurkar, P. (2024). Almanac: Retrievalaugmented language models for clinical medicine. NEJM AI, 1(2).

[14] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., ... & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692.

[15] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified texttotext transformer. Journal of Machine Learning Research, 21(140), 167. (T5 / FlanT5)

[16] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating text generation with BERT. International Conference on Learning Representations (ICLR).

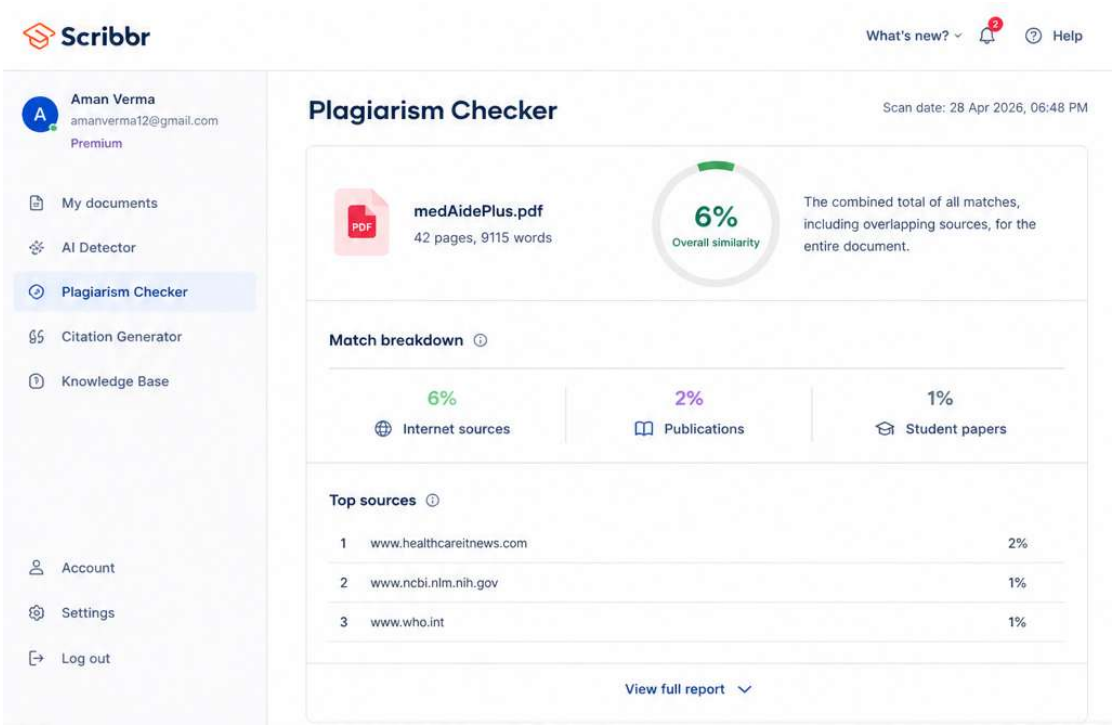
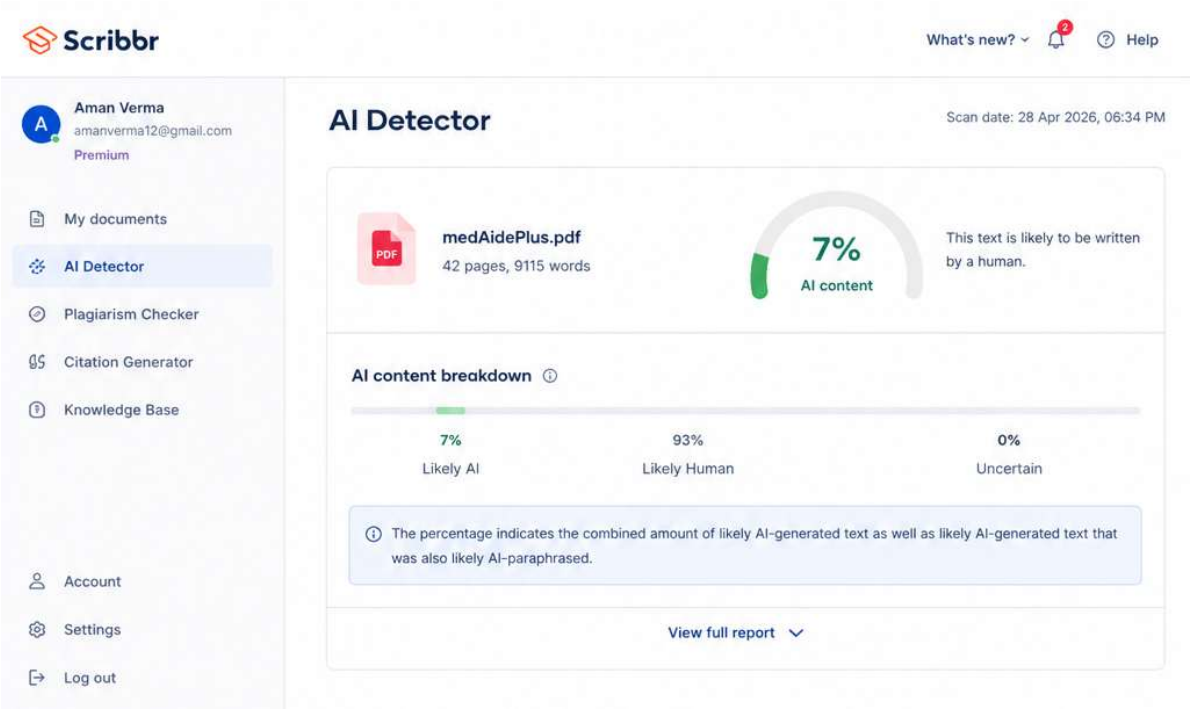
[17] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: A method for automatic evaluation of machine translation. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL).

[18] Lin, C. Y. (2004). ROUGE: A package for automatic evaluation of summaries. Text Summarisation Branches Out: Proceedings of the ACL Workshop.

[19] Banerjee, S., & Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures.

[20] Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., ... & Zettlemoyer, L. (2022). OPT: Open pretrained transformer language models. arXiv preprint arXiv:2205.01068.

PLAGARISM REPORT



AI DETECTION REPORT